

prompt 1

System / Role Context: Act as a Senior AI Engineer and a Lead IoT Backend Developer. Your task is to help me design and build a tool called "IoT-DocGen".

Project Context: "IoT-DocGen" is an automated API documentation generator for IoT devices. Instead of relying on rigid, rule-based parsing, this tool will use an LLM API (specifically the Gemini API) at runtime. The pipeline will be:

1. Read raw input files (e.g., messy JSON configs, raw code snippets, or basic endpoint descriptions of an IoT device).
2. Construct a prompt containing this raw data.
3. Send it to the LLM API.
4. Receive a beautifully structured, developer-friendly Markdown documentation (similar to Swagger/OpenAPI standards) and save it to a file.

Task: For this first step, I do NOT want any code yet. I want you to act as the software architect and provide a high-level technical plan for this tool.

Please provide:

1. **Architecture Overview:** A brief textual diagram or flow of how the Python script will work.
2. **Input/Output Definition:** What should the raw mock data look like for testing, and what should the final Markdown structure include?
3. **Step-by-Step Implementation Plan:** How we will build this together in our upcoming conversation turns.

Constraints:

- Do not write the Python code yet.
- Focus on logical separation of concerns (Reading file -> Prompt building -> API call -> Saving output).
- Think step-by-step.

prompt 2

This looks fantastic. The pipeline is clear, and the separation of concerns is exactly what I need. Let's execute **Step 1: Project Scaffolding & Input Reader**.

First, please provide the complete, copy-pasteable content for the three mock data files (`device_config.json`, `handler_snippet.py`, `extra_notes.txt`) so I can create a `mock_data` folder and save them locally.

Next, write the Python code for the Input Reader module (e.g., `input_reader.py`).

Requirements for the code:

- Create a function (e.g., `read_device_inputs(folder_path)`) that reads all relevant files from a given directory.
- It should handle potential errors gracefully (e.g., what if the folder doesn't exist, or a file is unreadable?).
- It should return a single, cleanly formatted string (or a dictionary) containing all the raw data, ready to be passed to the Prompt Builder in the next step.

Constraints:

- Do NOT write the Prompt Builder or API Client logic yet.
- Stick strictly to the reading and normalization logic.
- Include basic type hints and docstrings for clean code.

prompt 3

This is absolutely perfect. The use of explicit delimiters (--- BEGIN FILE ---) in `format_inputs_for_prompt` is excellent for LLM context window isolation.

Let's move to Step 2: Prompt Builder.

This is the core "Prompt Engineering" phase of the tool. I need you to write the `prompt_builder.py` module.

Requirements for the module:

Create a function (e.g., `build_documentation_prompt(raw_context_string: str) -> str`) that takes the normalized string from Step 1 and injects it into a carefully crafted LLM prompt template.

The Prompt Template itself is the most important part here. It must include:

Persona: Set the LLM to act as an Expert Technical Writer and Senior API Architect.

Task: Clearly define what the LLM needs to do (analyze the messy context and write OpenAPI/Swagger-like documentation).

Output Format Rules: Enforce strict Markdown formatting. Specify exactly what sections must exist (e.g., Overview, Authentication, Endpoints, Parameters Table, Error Codes).

Inference Guidelines: Tell the LLM how to handle missing data (e.g., "If rate limits are mentioned in notes, apply them to the relevant endpoints").

Return the final, fully constructed prompt string.

Constraints:

Do NOT write the LLM API Client (Step 3) yet. We are strictly focusing on assembling the string payload.

Ensure the prompt strictly forbids the LLM from outputting conversational filler (like "Here is your documentation...") – it should ONLY output the raw Markdown.

prompt 4

Brilliant. The prompt template is solid, especially the explicit synthesis rules and the strict negative constraints against conversational filler.

Let's execute **Step 3: Gemini API Client**.

I want to use the Google Gemini API (specifically the `gemini-1.5-pro` or `gemini-1.5-flash` model) to process our prompt. Please write the `llm_client.py` module.

Requirements for the code:

1. Use the official `google-generativeai` Python SDK.
2. Create a function (e.g., `generate_documentation(prompt_string: str) -> str`) that sends the prompt to the model and returns the generated text.
3. Use `python-dotenv` to load the `GEMINI_API_KEY` from a `.env` file (best practice for security).
4. Include robust error handling (e.g., catching API quotas, network issues, or authentication errors) and log them appropriately instead of crashing ungracefully.
5. Provide the exact `pip install` command needed for the requirements.

Constraints:

- Do NOT write the final Output Writer or `main.py` CLI logic yet (Step 4).
- Keep it modular so that the API call is fully abstracted.

prompt 5

Outstanding work. The error handling and the choice of `temperature=0.1` for deterministic output are exactly what distinguishes a professional pipeline from a toy script.

Let's execute the final steps: **Step 4 (Output Writer & CLI)** and **Step 5 (Orchestration)**.

Please write the final two Python files to complete our project:

1. **output_writer.py**: A simple module that takes the generated Markdown string and saves it to a specified file path.
 - It must ensure that the target directory exists (creating it if necessary).
 - Handle basic file I/O errors.
2. **main.py**: The main orchestrator script.
 - Use Python's built-in `argparse` module to create a standard CLI interface. It should accept `--input` (defaulting to `./mock_data`) and `--output` (defaulting to `./docs/device_api.md`).
 - Import the specific functions we created in `input_reader.py`, `prompt_builder.py`, `llm_client.py`, and `output_writer.py`.
 - Wire the pipeline together sequentially: *Read Input -> Normalize -> Build Prompt -> Generate Docs via API -> Save to File*.
 - Add clear, user-friendly terminal feedback (using `logging` or `print`) so the user can see the pipeline progressing step-by-step.

Constraints:

- Ensure all imported function names match exactly what you provided in the previous steps (`read_device_inputs`, `format_inputs_for_prompt`, `build_documentation_prompt`, `generate_documentation`).
- Provide instructions on how to run the final script from the terminal.